INDICTRUST

Development Roadmap

A phased plan to validate, build, and test a multilingual AI reliability tool—with an emphasis on reproducible linguistic failures, actionable diagnostics, and verified regression tests.

Initial focus Hindi-English code-switching and Romanized Hindi	**Primary users** Developers building multilingual AI assistants
Core workflow Generate → Test → Diagnose → Verify → Retest	**Estimated duration** Approximately 15–17 weeks, part-time

Guiding principle

We will not call a feature a competitive advantage until experiments show that it helps users do something meaningfully better than their existing workflow.

Version 1.0 • Prepared 14 October 2026

1. Executive Summary

IndicTrust is a proposed multilingual AI reliability testing and diagnostics tool. Its goal is to help developers discover where AI assistants fail on linguistically varied inputs, reproduce those failures, and prevent known failures from returning after model or prompt changes.

The core workflow

GENERATE	TEST	DIAGNOSE	VERIFY	RETEST

Product hypothesis: For a narrowly defined class of multilingual AI tasks, IndicTrust can make language-specific failures easier to reproduce, explain, and turn into regression tests than a baseline evaluation workflow. This is a hypothesis to test—not a proven market gap.

Initial scope

- Start with customer-support scenarios involving Hindi-English code-switching and Romanized Hindi.
- Build a small, human-reviewed benchmark before developing a full interface.
- Use existing models and tools where practical; do not train a foundation model from scratch.
- Compare against existing evaluation workflows before making novelty or superiority claims.
- Prioritize evidence, reproducibility, and useful diagnostics over feature count.

What success looks like

- The benchmark contains semantically matched examples with documented review status.
- Evaluation runs are reproducible and individual failures can be inspected.
- Diagnostics identify controlled, known failure patterns and link to supporting cases.
- Verified failures can be saved and automatically retested across versions.
- A fair comparison establishes whether the workflow provides measurable practical value.

2. Product Definition and Differentiation

Problem statement

An AI assistant may respond correctly to a canonical input but fail when the same intent is expressed through transliteration, code-switching, informal wording, or spelling variation. Developers need a repeatable way to identify such failures, understand their patterns, and check whether changes actually fix them.

Four intended product characteristics

Characteristic	Planned behavior	Evidence to seek
1. Controlled linguistic variation	Generate or manage variants across language, script, transliteration, code-switching, register, and spelling.	Variants preserve the intended meaning; ambiguous examples are flagged for review.
2. Failure diagnosis	Group failures by task and linguistic pattern, and link every diagnosis to actual test cases.	Known injected failures are detected; false alarms and uncertain cases are reported.
3. Verified regression tests	Allow a human to confirm a failure and save it as a reusable test.	The same failure can be reproduced across later model or prompt versions.
4. Evidence-based comparison	Compare versions using the same test set and show category-level improvements and regressions.	Results are repeatable and claims account for sample size and uncertainty.

Competitive reality: Multilingual testing, LLM evaluation, tracing, and regression workflows already exist in other tools. IndicTrust's differentiation must come from a demonstrably more useful focused workflow, not from claiming that these broad capabilities are new.

3. Phased Development Plan

PHASE 0 • WEEK 1

Competitive validation and problem definition

Objective: Determine whether the proposed differentiation is real enough to justify implementation.

Work items

- Inspect Promptfoo's multilingual testing and adversarial-testing capabilities.
- Inspect Langfuse's datasets, evaluations, experiments, and version-comparison workflows.
- Review relevant Indic-language benchmarks, including IndicGenBench.
- Record existing capabilities, limitations, and evidence for any suspected gap.
- Define a narrow hypothesis and a small experiment that could disprove it.

Deliverable: A competitor matrix, target-user statement, testable product hypothesis, and experiment plan.

Exit condition: Proceed only if there is a specific, testable workflow worth investigating; otherwise refine the scope.

PHASE 1 • WEEKS 2–3

Build the first multilingual test benchmark

Objective: Create a small, structured set of semantically matched customer-support inputs.

Work items

- Start with Hindi-English code-switching and Romanized Hindi; do not target every language at launch.
- Define fields for intent, canonical input, variant, transformation, expected behavior, review status, and notes.
- Create an initial target of roughly 100–200 cases only if they can be reviewed to a useful standard.
- Write a Python validator for required fields, malformed examples, and inconsistent metadata.
- Document how semantic equivalence and ambiguity will be reviewed.

Deliverable: Version-controlled dataset plus a repeatable validation script.

Exit condition: Dataset loads and validates; examples have traceable review status and a documented quality process.

PHASE 2 • WEEKS 4–5

Build the evaluation engine

Objective: Run a benchmark against a target system and generate reproducible evaluation records.

Work items

- Support a mock assistant or controlled customer-support application before paying for external APIs.
- Record input, output, model/prompt configuration where available, timestamp, and execution metadata.
- Implement task-appropriate evaluators and category-level metrics.
- Store results and produce a report with aggregate scores and individual cases.
- Test error handling, repeatability, and metric behavior.

Deliverable: A command-line evaluator that can rerun a benchmark and inspect individual results.

Exit condition: Runs produce comparable records, with documented configuration and visible per-case outcomes.

PHASE 3 • WEEKS 6–8

Build the linguistic failure analyzer

Objective: Find recurring failure patterns associated with linguistic variation.

Work items

- Compare canonical inputs with Romanized and code-switched variants.
- Group results by transformation and task category.
- Begin with transparent rules and statistical summaries before adding an LLM judge.
- Link each diagnostic to the test cases that support it.
- Test against deliberately introduced, known failure patterns.

Deliverable: A diagnostic report that surfaces failure categories and supporting examples.

Exit condition: Known controlled failure patterns are detected without treating every unusual input as a failure.

PHASE 4 • WEEKS 9–10

Build verified regression testing

Objective: Turn confirmed failures into reusable tests for later model or prompt versions.

Work items

- Let a reviewer confirm or reject a suspected failure.
- Store the input, expected behavior, actual output, category, and version metadata.
- Save confirmed cases as regression tests.
- Run the same tests against a later version and compare outcomes.
- Keep automatically suggested cases distinct from human-verified tests.

Deliverable: A working failure-to-regression-test workflow.

Exit condition: A controlled failure can be recorded, reproduced, and detected when it returns.

PHASE 5 • WEEKS 11–12

Build the developer API and interface

Objective: Make the evaluation workflow usable without manually editing files or records.

Work items

- Expose core operations through FastAPI.
- Use Streamlit for a simple initial interface.
- Support selecting test suites, running evaluations, filtering failures, and comparing runs.
- Display individual test evidence and reviewer decisions.
- Add tests and package the application reproducibly.

Deliverable: A local application that supports the complete workflow through a developer-facing interface.

Exit condition: A user can run evaluations, inspect findings, verify failures, and save regression tests through the interface.

PHASE 6 • WEEKS 13–15

Benchmark against existing approaches

Objective: Measure whether IndicTrust provides an advantage on the chosen task.

Work items

- Compare a baseline workflow, IndicTrust, and an existing tool where feasible.
- Use the same target system and comparable test cases.
- Measure detection quality, diagnostic correctness, false positives, review effort, and reproducibility.
- Test whether regression checks catch a real, controlled deterioration.
- Document limitations and avoid claims unsupported by the experiment.

Deliverable: A reproducible benchmark report with methodology, results, examples, and limitations.

Exit condition: Evidence supports a meaningful advantage on a defined task—or indicates that the product hypothesis must change.

PHASE 7 • INITIAL 2-WEEK PILOT

Test with real developers

Objective: Learn whether the workflow solves a practical problem for target users.

Work items

- Recruit developers working on multilingual AI assistants where possible.
- Observe their current evaluation process and tools.
- Ask them to test the workflow on representative tasks.
- Record whether it finds actionable issues, saves time, and fits their workflow.
- Prioritize improvements based on evidence rather than feature requests alone.

Deliverable: A pilot report with user feedback, observed value, limitations, and next steps.

Exit condition: At least one concrete workflow shows practical value; otherwise revisit scope and positioning.

4. Proposed Technical Stack

Layer	Initial choice	Reason / timing
Core language	Python	Primary implementation language and broad ML ecosystem.
Data handling	pandas, NumPy	Dataset validation, transformation, and analysis.
ML utilities	scikit-learn; custom evaluators	Baselines, conventional ML metrics, and controlled experiments.
API	FastAPI	Introduce once the evaluation engine has stable interfaces.
Storage	SQLite first; PostgreSQL later	Keep early development simple; upgrade when concurrent users or deployment needs justify it.
Testing	pytest	Automated checks for data validation, evaluation, diagnostics, and regression behavior.
Interface	Streamlit initially	Quick way to expose useful workflows before investing in a custom frontend.
Version control	Git and GitHub	Reviewable history, reproducibility, issues, and portfolio evidence.
Packaging	Docker later	Add when the application workflow is stable enough to package.

Architecture principle: keep the first version modular but small. Avoid adding cloud infrastructure, a complex frontend, or paid model services until they are necessary to test a product hypothesis.

5. Skill Development and Portfolio Evidence

Skill	Where it develops	Evidence to publish
Python engineering	Phases 1–5	Readable modules, type-aware interfaces where useful, and tested code.
Data handling	Phases 1–3	Validated dataset schema and data-quality report.
ML/LLM evaluation	Phases 2–3, 6	Metric definitions, per-case evidence, and repeatable evaluation runs.
Statistical reasoning	Phases 3 and 6	Controlled comparisons, uncertainty notes, and honest limitations.
API and software testing	Phases 2–5	Documented API, automated tests, and reproducible setup.
Version control and delivery	Throughout	Meaningful commits, README, setup instructions, and release notes.
Technical communication	Throughout	Architecture decisions, experiment reports, and a demo.

6. Validation Metrics and Guardrails

We should evaluate both the target AI system and IndicTrust itself. A tool that produces more alerts is not necessarily better; its findings must be correct, reproducible, and useful.

Measure	What it tells us	Caution
Failure detection	Whether known failures are surfaced.	A controlled test set may not represent all real-world failures.
Diagnostic correctness	Whether assigned categories and explanations match reviewed evidence.	Use human review on a representative sample.
False-positive rate	How often acceptable behavior is incorrectly flagged.	Define acceptable behavior before running tests.
Reproducibility	Whether results can be recreated under documented conditions.	Model APIs may be nondeterministic or change over time.
Review effort	Human time required to verify cases and act on findings.	Track time consistently across comparison workflows.
Regression detection	Whether known failures return after a version change.	Use controlled changes and stable test expectations.
User value	Whether developers find actionable issues or save time.	Positive feedback alone is not proof of willingness to pay.

Scope guardrails

- Do not claim uniqueness or superiority without comparative evidence.
- Do not treat generated variants as valid until their intended meaning is checked.
- Separate automatically suspected failures from human-verified failures.
- Do not equate data drift with degraded model performance.
- Do not use one aggregate score to hide regressions in a linguistic subgroup.
- Keep model, prompt, dataset, evaluator, and run configuration recorded wherever feasible.
- Prefer a small reliable benchmark over a large unreviewed synthetic dataset.

7. Immediate First Milestone

Question to test: Can we identify a reproducible difference in how a target AI system handles semantically equivalent English, Romanized Hindi, and Hindi-English code-switched inputs—and explain that difference accurately?

First experiment checklist

- Choose one narrow customer-support task with clear expected behavior.
- Create a small set of semantically matched inputs in the three initial input styles.
- Record the intended meaning and review status of each variant.
- Run the same target system with consistent settings and preserve raw outputs.
- Review failures manually and separate genuine failures from evaluator mistakes.
- Inspect what an existing evaluation tool can already detect or report.
- Write a short result summary before deciding which feature to implement next.

Decision rule: if the experiment does not reveal a meaningful gap, revise the product hypothesis before building more software. Finding that an idea is weak early is a successful validation outcome.

8. Key Risks and Mitigations

Risk	Mitigation
Existing tools already cover the proposed workflow.	Run competitive validation first; differentiate on a measurable narrow workflow or change direction.
Synthetic variants change the meaning.	Track transformations, use human review, and flag ambiguity instead of assuming equivalence.
Automated evaluators produce false diagnoses.	Use transparent rules where possible, retain supporting evidence, and sample cases for human review.
Scope grows too quickly.	Start with one task and two linguistic variation types; add scope only after validation.
Project looks like a tutorial rather than engineering work.	Publish tests, reproducible experiments, architecture decisions, baselines, and honest results.
No one is willing to adopt or pay for it.	Interview target developers and run a small pilot before committing to a larger product.

9. Initial References

These resources are starting points for competitive validation. Their capabilities and documentation may change; record the date and exact workflow tested during Phase 0.

• **Promptfoo — Red-team configuration and multilingual testing**
https://www.promptfoo.dev/docs/red-team/configuration/

• **Langfuse — Offline evaluation**
https://langfuse.com/docs/evaluation/get-started/offline

• **Google Research — IndicGenBench**
https://github.com/google-research-datasets/indic-gen-bench

Final principle

Build evidence first, then features. IndicTrust earns its place as a product only if it helps developers find, understand, and prevent meaningful multilingual AI failures more effectively than their existing workflow.